

ASSIGNMENT STATIC UNEVEN DISTRIBUTION (EVEN NRP)	Nama: Farhan Kamil Hermansyah NRP: 152024150 Mata Kuliah: IFB 206 - Komputasi Paralel dan Sistem Terdistribusi Dosen Pengampu: Lisa Kristiana PhD
---	--

Github :

https://github.com/Farmil23/IFB-206-KOMPUTASI-PARALEL-SISTEM-TERDISTRIBUSI/tree/main/LECTURE_7_QUIZ

1. CODE PYTHON STUDI KASUS *DISTRIBUSI REQUEST KE CLUSTER SERVER HETEROGEN*

```
import time

def simulasi_static_distribution_150():
    # Studi Kasus: Distribusi Request ke Cluster Server Heterogen
    # Server A (High-Spec), Server B (Mid-Spec), Server C (Low-Spec)
    print("=== MONITORING DISTRIBUSI RESOURCE - NRP 150 ===")

    tasks = 150 # Total beban kerja berdasarkan NRP
    nodes = {
        "Server_A_High": {"weight": 0.6, "allocated": 0},
        "Server_B_Mid": {"weight": 0.3, "allocated": 0},
        "Server_C_Low": {"weight": 0.1, "allocated": 0}
    }

    # Menghitung target ideal berdasarkan bobot statis
    targets = {node: int(tasks * data["weight"]) for node, data in nodes.items()}

    print(f"Target Distribusi Ideal: {targets}\n")
    print("Memulai proses alokasi task...")
    print("-" * 45)

    # Proses alokasi deterministik
    for i in range(1, tasks + 1):
        if nodes["Server_A_High"]["allocated"] < targets["Server_A_High"]:
            nodes["Server_A_High"]["allocated"] += 1
        elif nodes["Server_B_Mid"]["allocated"] < targets["Server_B_Mid"]:
            nodes["Server_B_Mid"]["allocated"] += 1
        else:
            nodes["Server_C_Low"]["allocated"] += 1

    # Cek apakah distribusi ideal telah tercapai (Convergence Point)
    current_alloc = {k: v["allocated"] for k, v in nodes.items()}
    if current_alloc == targets:
        print(f"STEP {i}: [!] IDEAL DISTRIBUTION REACHED")
        break

    # Tampilkan log per 30 task untuk efisiensi layar
    if i % 30 == 0:
        print(f"Step {i}: Current State -> {current_alloc}")

    print("-" * 45)
    print("FINAL REPORT (COMPLIANCE CHECK):")
```

```

for node, target in targets.items():
    actual = nodes[node]["allocated"]
    status = "MATCH" if actual == target else "MISMATCH"
    print(f'{node}: Target {target} | Actual {actual} -> {status}')

if __name__ == "__main__":
    simulasi_static_distribution(150)

```

2. OUTPUT CODE

```

D:\LIFE\KULIAH_ITENAS\SEMESTER 4\KOMPUTASI PARALLEL\LECTURE_9>python main.py
=== MONITORING DISTRIBUSI RESOURCE - NRP 150 ===
Target Distribusi Ideal: {'Server_A_High': 90, 'Server_B_Mid': 45, 'Server_C_Low': 15}

Memulai proses alokasi task...
-----
Step 30: Current State -> {'Server_A_High': 30, 'Server_B_Mid': 0, 'Server_C_Low': 0}
Step 60: Current State -> {'Server_A_High': 60, 'Server_B_Mid': 0, 'Server_C_Low': 0}
Step 90: Current State -> {'Server_A_High': 90, 'Server_B_Mid': 0, 'Server_C_Low': 0}
Step 120: Current State -> {'Server_A_High': 90, 'Server_B_Mid': 30, 'Server_C_Low': 0}
STEP 150: [!] IDEAL DISTRIBUTION REACHED
-----
FINAL REPORT (COMPLIANCE CHECK):
Server_A_High: Target 90 | Actual 90 -> MATCH
Server_B_Mid: Target 45 | Actual 45 -> MATCH
Server_C_Low: Target 15 | Actual 15 -> MATCH

D:\LIFE\KULIAH_ITENAS\SEMESTER 4\KOMPUTASI PARALLEL\LECTURE_9>

```

3. ANALISIS TERHADAP DISTRIBUTION PADA OUTPUT

Dalam arsitektur sistem terdistribusi, skenario **Static Uneven Distribution** adalah implementasi dari strategi *Capacity-Aware Scheduling*. Berikut adalah analisis teknisnya:

A. Paradigma Heterogeneous Cluster

Dalam jaringan skala industri yang sudah besar ya, biasanya jarang ditemukan server dengan spesifikasi yang sama secara 100% atau sama persis. Nah, studi kasus ini akan mensimulasikan lingkungan misalkan saya memiliki server dengan kapasitas CPU/RAM yang berbeda dan memiliki kemampuan berbeda. Dalam konteks ini, simulasi yang saya lakukan menggunakan NRP saya sendiri yaitu 150 sebagai total beban kerja bertujuan untuk merefleksikan manajemen sumber daya yang presisi melalui metode *Static Uneven Distribution*, sebuah strategi yang secara teknis sangat krusial dalam mengoptimalkan *Return on Investment* (ROI) infrastruktur dengan cara memberikan bobot statis sebesar 0.6 kepada Server A sebagai node paling *powerful*, 0.3 kepada Server B untuk performansi menengah, dan 0.1 kepada Server C yang memiliki spesifikasi paling rendah.

Pendekatan pembagian beban kerja yang asimetris ini bukan hanya untuk pembagian tugas kecil atau biasa aja, tapi sebuah implementasi dari prinsip **Capacity-Aware Scheduling** yang bertujuan untuk memastikan bahwa setiap siklus jam CPU pada server mahal seperti Server A dimanfaatkan secara maksimal guna mencegah terjadinya *idle time* yang merugikan secara ekonomi, sekaligus melindungi server dengan spesifikasi rendah seperti Server C dari risiko *overload* yang dapat memicu kegagalan sistem atau degradasi layanan.

Secara algoritmik, kode yang saya susun memantau setiap tahap alokasi hingga sistem mencapai titik *Ideal Distribution* dengan sebuah kondisi *Steady State* di mana rasio beban kerja terhadap kapasitas di seluruh node telah terdistribusi secara proporsional sesuai parameter bobot yang ditentukan sehingga fluktuasi *throughput* dapat diminimalisir dan waktu respon aplikasi menjadi lebih konsisten bagi pengguna akhir.

Dengan menerapkan logika ini, seorang *Network Engineer* atau *AI Engineer* yang paling saya alami ya agar dapat menjamin bahwa aliran data dalam jaringan terdistribusi secara deterministik, memudahkan proses monitoring, serta menyediakan fondasi yang stabil bagi sistem untuk melakukan *scaling* di masa depan tanpa harus mengorbankan stabilitas operasional pada node-node yang lebih lemah, yang pada akhirnya menciptakan sebuah orkestrasi infrastruktur yang resilien, efisien, dan mampu menangani lonjakan trafik secara terukur sesuai dengan profil kapasitas riil dari masing-masing unit pemrosesan di lapangan.

B. Analisis Ideal Distribution (Convergence)

```
Memulai proses alokasi task...
-----
Step 30: Current State -> {'Server_A_High': 30, 'Server_B_Mid': 0, 'Server_C_Low': 0}
Step 60: Current State -> {'Server_A_High': 60, 'Server_B_Mid': 0, 'Server_C_Low': 0}
Step 90: Current State -> {'Server_A_High': 90, 'Server_B_Mid': 0, 'Server_C_Low': 0}
Step 120: Current State -> {'Server_A_High': 90, 'Server_B_Mid': 30, 'Server_C_Low': 0}
STEP 150: [!] IDEAL DISTRIBUTION REACHED
```

Pada hasil output, Implementasi ini melampaui pembagian data sederhana dengan secara aktif memonitor status sistem terhadap *target state*. **Ideal Distribution** tercapai tepat saat rasio beban kerja berbanding lurus dengan kapasitas perangkat keras. Dalam domain AI, titik ini dikenal sebagai **Convergence Point**, di mana fungsi distribusi telah memenuhi seluruh batasan (*constraints*) tanpa menyisakan varians. Sistem secara otomatis menghentikan iterasi ketika harmoni antara bobot dan alokasi tercapai. Akhirnya, prosedur *Compliance Check* pada laporan final memastikan bahwa *actual distribution* identik dengan target matematis, menjamin akurasi distribusi beban pada infrastruktur server heterogen secara presisi dan deterministik.

Alur Untuk Kode tersebut seperti ini



